

Zendex RewardsEngine Unbounded userTokens Array Growth

Permissionless griefing vector causing reward-claim gas exhaustion

13.6M	61	60
gas used by claim()	total tracked entries	attacker-injected entries

Target: HackenProof public Zendex smart-contract repository

Finding: Anyone could force-grow another user's RewardsEngine.userTokens array through ordinary permissionless swaps by setting the victim as the swap recipient.

Research date: 25 August 2026

Portfolio presentation of an independent security research submission. This document summarizes the research, reproduction, impact analysis, and final triage outcome; it is not a client audit report.

1. Executive Summary

I identified a permissionless griefing vector in Zendex's RewardsEngine where an attacker could cause another user's userTokens array to grow without the victim's participation. The attacker did this by creating throwaway ERC20 tokens and pairs, then using ordinary router swaps with the victim supplied as the to recipient.

The underlying problem was an interaction between two behaviors: ZendexRouter._distributeFees() credited rewards to the caller-supplied recipient, while RewardsEngine.claim() processed the entire userTokens[msg.sender] array in one atomic call. There was no user-controlled subset selection or pruning path.

A full Foundry reproduction against the real, unmodified ZendexFactory, ZendexRouter, and RewardsEngine contracts grew the victim's array from 1 legitimate entry to 61 entries using 60 attacker-controlled transactions. The measured claim() call consumed 13,641,509 gas.

The original submission described this as a permanent freeze of cashback and burn rewards once the array exceeded the practical block-gas ceiling. During triage, HackenProof confirmed the mechanism and the asymmetry, but closed the report as out of scope / known issue under F-2026-16633. The triage note also stated that the external-trigger model moves the issue from the original Informational framing into Low on the narrow scope and Medium when accounted for, and that the audit recommendation included paged claiming.

Key evidence

Target	hackenproof-public/zendex-sc
Report	ZNDXSCDD-96
Created	25 August 2026
PoC	Foundry — compiled and passing against real contracts
Measured state	1 legitimate + 60 injected = 61 entries
Measured gas	13,641,509 gas for claim() with 61 entries
Final triage	Mechanism confirmed; closed as out of scope / known issue; no reputation penalty

2. Technical Root Cause

The finding emerged from tracing how RewardsEngine learns which tokens belong to a user.

2.1 Caller-controlled reward recipient

ZendexRouter._distributeFees() passes the swap's p.to parameter as trader:

```
_distributeFees(p.path[0], p.to, p.amountIn);

function _distributeFees(address token, address trader, uint256 amountIn)
    internal returns (uint256 amountToPair)
{
    ...
    IRewardsEngine($main.rewardsEngine).depositFees(
        token, trader, treasuryFee, cashbackFee, burnFee
    );
    ...
}
```

Because p.to is supplied by the swap caller, the address receiving the swap output can be selected by someone other than that address. HackenProof noted that third-party crediting is not inherently a defect; the issue arises from the downstream storage and claimability design.

2.2 Unconditional token tracking

```
function _trackUserToken(address user, address token) internal {
    if (!$.userHasToken[user][token]) {
        $.userTokens[user].push(token);
        $.userHasToken[user][token] = true;
    }
}
```

An attacker can therefore add distinct token addresses to a victim's array without requiring the victim to approve, sign, or initiate anything.

2.3 Claim processes the whole array

```
function claim(uint256 minCashbackOutput, uint256 minBurnOutput, uint256 deadline)
    external nonReentrant returns (uint256 totalNative)
{
    address[] memory tokens = $.userTokens[msg.sender];
    ...
    for (uint256 i; i < tokens.length; ++i) { ... }
    ...
}
```

At the time of submission, claim() did not accept a token subset, and the report identified no pruning path. This created the central asymmetry: an attacker could append entries incrementally across many transactions, while the victim's claim had to process the accumulated array atomically.

3. Attack Path

The PoC deliberately used ordinary public functionality rather than privileged access or protocol-internal shortcuts.

01 Establish a legitimate victim state

The victim performs one normal WZEN/USDT trade and receives genuine reward tracking. The test confirms `userTokens[victim].length == 1`.

02 Create attacker-controlled token pairs

For each spam iteration, the attacker deploys a throwaway ERC20 and creates a pair with WZEN through the public factory.

03 Seed minimal liquidity

The attacker adds a small amount of liquidity to make the pair usable by the router.

04 Route the swap to the victim

The attacker calls `swapExactTokensForTokens` with `to: victim`. The victim does not participate.

05 Repeat with fresh tokens

Each distinct token produces another `userTokens` entry. The PoC performs 60 iterations.

06 Measure victim claim gas

The victim calls `claim()`. With 61 entries, the measured gas consumption is 13,641,509.

Attack chain

```
permissionless swap → attacker chooses victim as p.to → depositFees(victim, ...) →  
_trackUserToken(victim, freshToken) → userTokens[victim] grows → claim() iterates  
every entry
```

The PoC assertion `poisonedCount == legitimateEntryCount + spamCount` demonstrates that all 60 attacker iterations added entries to the victim's array.

4. Proof of Concept

The supplied Foundry test was described as compiled and passing against the real, unmodified Zendex contracts. It used the actual ZendexFactory, ZendexRouter, RewardsEngine, WZEN, and proxy initialization paths.

Test result

```
forge Version: 1.7.1 (4072e48 2026-05-08)
[PASS] test_attackerForceGrowsVictimUserTokensArray_freezingRealRewards()
```

```
Victim's userTokens array length after ONE real, legitimate trade: 1
Victim's userTokens array length after attacker's spam: 61
Gas consumed by claim() with 61 entries: 13641509
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The PoC avoided mocks for the vulnerable execution path. The setup deploys the relevant contracts and uses the public factory/router flow for attacker-created pairs and swaps.

Discovery path

The research notes state that static analysis highlighted the costly loop in RewardsEngine.claim(). Tracing what controlled the loop bound led from userTokens[msg.sender] to _trackUserToken() and then to depositFees(), exposing the caller-controlled trader/recipient path.

Reproduction command

```
forge test --match-contract RewardsEngineUserTokensDosTest -vv
```

The spam count was intentionally kept at 60 so the reproduction remained practical while still producing a directly measured gas result.

5. Impact Analysis

The security property at risk was the victim's ability to execute `RewardsEngine.claim()` within a transaction's available gas budget.

Dimension	Observed / reported effect
Attacker permission	No privileged role; ordinary permissionless factory, liquidity, and router operations.
Victim participation	None required for poisoning transactions.
Attacker-controlled state	Number of distinct token entries in the victim's <code>userTokens</code> array.
Measured point	61 entries → 13,641,509 gas for <code>claim()</code> .
Scaling	Report observed approximately linear gas growth with array size.
Funds	Cashback/burn balances were described as remaining in <code>RewardsEngine</code> rather than being directly stolen.
Primary consequence	Potential inability to execute the all-entries claim once gas consumption exceeds the practical block limit.

Gas extranolation

The submitted PoC measured approximately 224K gas per entry at the observed point. The report extrapolated that roughly 130–150 total entries could exceed a 30M-gas block ceiling. This is an extrapolation from the measured result, not a separately executed 130–150-entry test.

Triage clarification

HackenProof's September 11 response confirmed the mechanism and stated that the external-trigger model changed the assessment from the original Informational framing into Low on the narrow scope and Medium once accounted for. The response also stated that the underlying claim-design issue was recorded as F-2026-16633 and that the audit recommendation included paged claiming. The report was closed as out of scope / known issue and received no reputation penalty.

6. Remediation Considerations

The submitted report proposed several complementary ways to remove the attack surface or bound its effect.

A. Restrict reward crediting

For fee-crediting purposes, decouple reward attribution from an arbitrary swap recipient, or require explicit opt-in / authorization when a third party is intended to receive rewards.

B. Bound userTokens length

Introduce a maximum number of tracked token addresses per user, with a defined policy for rejection or eviction once the bound is reached.

C. Support partial / selective claiming

Allow a user to claim a caller-selected subset of tracked tokens, so a bloated array or problematic entry cannot block access to every other reward balance.

D. Paged claiming

HackenProof's triage response specifically referenced a paged claim taking a start and end index as the audit recommendation for the gas variant. This addresses the all-at-once iteration constraint by bounding work per transaction.

7. Research Methodology Demonstrated

This case study demonstrates a workflow centered on tracing economic state and execution constraints rather than treating static-analysis output as the finding itself.

Static-analysis signal: Aderyn's costly-operation-in-loop warning highlighted `RewardsEngine.claim()`; Slither also surfaced the claim function's reentrancy shape, which was not pursued because `nonReentrant` protected the call.

State-flow tracing: The investigation followed the loop bound backward from `userTokens[msg.sender]` to `_trackUserToken()` and then to `depositFees()`.

Trust-boundary analysis: The key question became who controls the trader/recipient supplied to `_distributeFees()`, and whether that address had consented to the state mutation.

Economic attack construction: The attacker was given only public factory/router capabilities and fresh token addresses were used to turn cheap actions into persistent victim-side state growth.

Empirical validation: Rather than relying solely on gas estimates, the PoC measured `claim()` directly after controlled array growth.

Triage-aware impact analysis: The final case study distinguishes the exploit mechanism from the narrower final assessment and known-issue status.

Research principle: **find the invariant, trace who can mutate it, then prove the economic consequence.**

8. Appendix — Selected Evidence

Core tracking logic

```
if (!$.userHasToken[user][token]) {  
    $.userTokens[user].push(token);  
    $.userHasToken[user][token] = true;  
}
```

Core claim iteration

```
address[] memory tokens = $.userTokens[msg.sender];  
  
for (uint256 i; i < tokens.length; ++i) {  
    ...  
}
```

Attacker-controlled recipient in the PoC

```
router.swapExactTokensForTokens(  
    IZendexRouter.SwapExactTokensForTokensParams({  
        amountIn: 100_000e18,  
        amountOutMin: 0,  
        path: junkPath,  
        to: victim,  
        deadline: block.timestamp + 1 hours  
    })  
);
```

PoC assertions

```
assertEq(legitimateEntryCount, 1, ...);  
assertEq(poisonedCount, legitimateEntryCount + spamCount, ...);
```

Final note

This portfolio report preserves the distinction between the submitted exploit framing and the final triage outcome. The mechanism was confirmed, but the submission was not rewarded because HackenProof treated the underlying claim-design issue as a known/out-of-scope condition. No client engagement is implied by this case study.

Firman Siregar
Web3 Security Researcher
Smart Contracts • DeFi • Protocol Risk